CSE485–CSE488 Deep Learning
Assignment 1: MicroGrad (Scalar Autograd Engine)

Mohamed Wael Badra
23P0059

November 3, 2026

1 Overview

The goal is a reverse-mode automatic differentiation engine that works on single scalars, together with a small neural network API built on top of it. Every arithmetic operation on a `Value` creates a new node that remembers its inputs and the operation that produced it. Calling `backward()` on the final node walks this graph in reverse and applies the chain rule at every node, so each leaf ends up holding $\partial L/\partial$leaf.

The code is split into two files, as required by the template:

- `src/engine.py`: the `Value` class, its operators, the activations `tanh` and `exp`, and the topological-sort backward pass.
- `src/nn.py`: `Neuron`, `Layer` and `MLP`.

2 The Computational Graph

2.1 The `Value` node

Each node stores:

- `data`: the scalar computed in the forward pass.
- `grad`: $\partial L/\partial$(this node), starting at 0.
- `_prev`: the set of input nodes (children in the graph).
- `_op`: a string naming the operation, used for inspection.
- `_backward`: a closure that moves this node's gradient into its children.

For a node $z = g(x_1, \ldots, x_k)$, the chain rule gives the contribution of z to each input:

$$\frac{\partial L}{\partial x_i} \mathrel{+}= \frac{\partial g}{\partial x_i} \cdot \frac{\partial L}{\partial z}. \tag{1}$$

The term $\partial L/\partial z$ is `out.grad`, and $\partial g/\partial x_i$ is the *local derivative*, which depends only on the operation. Each `_backward` closure implements exactly this line for one operation.

2.2 Local derivatives

Addition. $z = x + y$:

$$\frac{\partial z}{\partial x} = 1, \qquad \frac{\partial z}{\partial y} = 1 \;\Rightarrow\; x.\text{grad} \mathrel{+}= z.\text{grad}, \quad y.\text{grad} \mathrel{+}= z.\text{grad}.$$

Addition passes the gradient through unchanged to both inputs.

Multiplication. $z = xy$:

$$\frac{\partial z}{\partial x} = y, \qquad \frac{\partial z}{\partial y} = x \;\Rightarrow\; x.\text{grad} \mathrel{+}= y \cdot z.\text{grad}, \quad y.\text{grad} \mathrel{+}= x \cdot z.\text{grad}.$$

Each input's gradient is scaled by the value of the other input.

Power by a constant. $z = x^n$, $n \in \mathbb{R}$ fixed:

$$\frac{\partial z}{\partial x} = n\,x^{n-1} \;\Rightarrow\; x.\text{grad} \mathrel{+}= n\,x^{n-1} \cdot z.\text{grad}.$$

Hyperbolic tangent. $z = \tanh x = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$. By the quotient rule,

$$\frac{d}{dx}\tanh x = \frac{(e^x + e^{-x})^2 - (e^x - e^{-x})^2}{(e^x + e^{-x})^2} = 1 - \tanh^2 x,$$

so $x.\text{grad} \mathrel{+}= (1 - z^2) \cdot z.\text{grad}$. The derivative is written in terms of the output z, which is already stored, so no extra exponentials are computed in the backward pass.

Exponential. $z = e^x$:

$$\frac{d}{dx}e^x = e^x = z \;\Rightarrow\; x.\text{grad} \mathrel{+}= z \cdot z.\text{grad}.$$

2.3 Derived operations

The remaining operators are built from the ones above, so they need no backward code of their own:

$$-x = x \cdot (-1),$$
$$x - y = x + (-y),$$
$$x/y = x \cdot y^{-1}.$$

For division the gradient follows automatically: $\partial(xy^{-1})/\partial y = x \cdot (-1)\,y^{-2} = -x/y^2$, which is the quotient rule. Dividing by zero raises `ZeroDivisionError` because Python refuses to compute 0.0^{-1}.

The reflected operators `__radd__`, `__rmul__`, `__rsub__` and `__rtruediv__` handle expressions such as `2.0 + a`, where the plain number is on the left.

2.4 Type handling

A helper `toValue` wraps `int` and `float` operands in a `Value` and raises `TypeError` for anything else. So `Value(2.0) + "string"` and `Value(2.0) * [1, 2, 3]` fail immediately with a clear message, instead of failing later or, for the list case, silently repeating the list.

The exponent of `__pow__` is asserted to be an `int` or `float`. Supporting a `Value` exponent would need the extra derivative $\partial(x^y)/\partial y = x^y \ln x$, which is undefined for $x \le 0$. Since the networks here only need constant powers, the engine rejects that case outright.

3 Backpropagation with a Topological Sort

3.1 Why an ordering is needed

A node's `_backward` reads `out.grad` and pushes it into the children. That is only correct if `out.grad` is final, meaning every node that uses `out` has already sent its contribution. Visiting nodes in reverse topological order guarantees this: in a topological order every node appears after all of its inputs, so in reverse every node appears before all of its inputs.

3.2 Building the order

```
def _get_topo(self):
    topo = []
    visited = set()

    def buildTopo(node):
        if node not in visited:
            visited.add(node)
            for child in node._prev:
                buildTopo(child)
            topo.append(node)

    buildTopo(self)
    return topo

def backward(self):
    topo = self._get_topo()
    self.grad = 1.0
    for node in reversed(topo):
        node._backward()
```

The search is a post-order depth-first search: a node is appended only after all of its children are appended. The root gets $\partial L/\partial L = 1$, and `_backward` then runs from the root down to the leaves.

3.3 The two traps

Post-order trap. If the node were appended *before* its children are visited (pre-order), a parent could appear in the list before a node that also depends on it through another path. In the reversed list that node would then run its `_backward` before its own gradient is complete, and the missing part would never reach the leaves.

Diamond trap. Consider $a = 3$, $c = a + a$, $d = c \cdot a$. The node a is reached by two paths, directly from d and through c. Without the `visited` set, a would be added to `topo` several times, and its `_backward` would run several times. That is harmless for a leaf, but if a were an inner node its children would receive the same contribution more than once. With the set, every node appears exactly once.

A related requirement is that gradients are *accumulated* with `+=`. For $c = a + a$, `_prev` is the set $\{a\}$, but `_backward` still writes to `self.grad` and `other.grad`, which are the same object. With `+=` both contributions add up:

$$d = 2a^2, \qquad \frac{\partial d}{\partial a} = \underbrace{c}_{\text{from } d=c\cdot a} + \underbrace{a \cdot (1+1)}_{\text{from } c=a+a} = 6 + 6 = 12 = 4a,$$

which matches the analytic result. With plain `=`, the backward pass of d first sets a.grad $= 6$, and then the backward pass of c overwrites it twice with 3, so the result would be 3.

3.4 Worked example

For $a = -2$, $b = 3$, $d = ab$, $e = a + b$, $f = de$:

Node	Value	Gradient
f	-6	1
d	-6	$e \cdot 1 = 1$
e	1	$d \cdot 1 = -6$
a	-2	$b \cdot \partial f/\partial d + \partial f/\partial e = 3 - 6 = -3$
b	3	$a \cdot \partial f/\partial d + \partial f/\partial e = -2 - 6 = -8$

These match $\partial f/\partial a = 2ab + b^2 = -3$ and $\partial f/\partial b = a^2 + 2ab = -8$, and they are the values printed by the notebook.

4 The Neural Network API

Neuron. A neuron with n inputs holds weights $w_1, \ldots, w_n$ and a bias b, all drawn from $\mathcal{U}(-1, 1)$, and computes

$$\text{out} = \tanh\Big(b + \sum_{i=1}^{n} w_i x_i\Big).$$

The sum is started from b (`sum(..., self.b)`), so the bias is part of the same graph and no extra addition node is needed.

Layer. A list of n_{out} independent neurons that all see the same input. When $n_{\text{out}} = 1$ the single `Value` is returned instead of a one-element list, so a network with one output can be used directly as a loss.

MLP. Given `nin` and a list `nouts`, the sizes $[\text{nin}, n_1, \ldots, n_k]$ define k consecutive layers; the output of each layer is the input of the next. The parameter count of a layer is $n_{\text{out}}(n_{\text{in}} + 1)$, so `MLP(3, [4, 4, 1])` has $16 + 20 + 5 = 41$ parameters, as the tests expect.

Parameters and zeroing. `parameters()` returns a flat list at every level, and `zero_grad()` calls `reset_grad()` on each one. Gradients must be cleared before every backward pass, because `+=` would otherwise add the new gradients onto the old ones.

5 Experiments

5.1 Unit tests and gradient checks

All 12 tests of the provided suite pass. In addition, the notebook compares autograd with central finite differences $\big(g(x+h) - g(x-h)\big)/2h$, $h = 10^{-6}$, on

$$g(x, y) = \tanh(xy + x^2) - e^{y/x} + 3 - \frac{2y^3}{1 + x^2},$$

which uses every operator. The two agree to about six decimal places at all test points.

5.2 Training on two moons

Setup: 100 points from `make_moons` (noise 0.1), labels mapped to ± 1. The model is `MLP(2, [16, 16, 1])`, which has 337 parameters. It is trained with full-batch gradient descent for 100 steps on

$$L = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2 + 10^{-4} \sum_j \theta_j^2,$$

with the learning rate decaying linearly from 1.0 to 0.1. A point is classified by the sign of $\hat{y}$.

	Step 0	Step 99
Loss	1.0976	0.0183
Accuracy	69.0%	100.0%

The loss falls by about a factor of 60 and every training point ends up on the correct side of the boundary. The random network already scores 69% because a roughly linear split separates most of the two moons; the remaining points sit where the moons interlock, and the curved boundary the network learns covers them as well. These numbers are training accuracy only. The goal here is to show that the engine's gradients drive the optimisation correctly, not to measure generalisation, so no test split is held out.

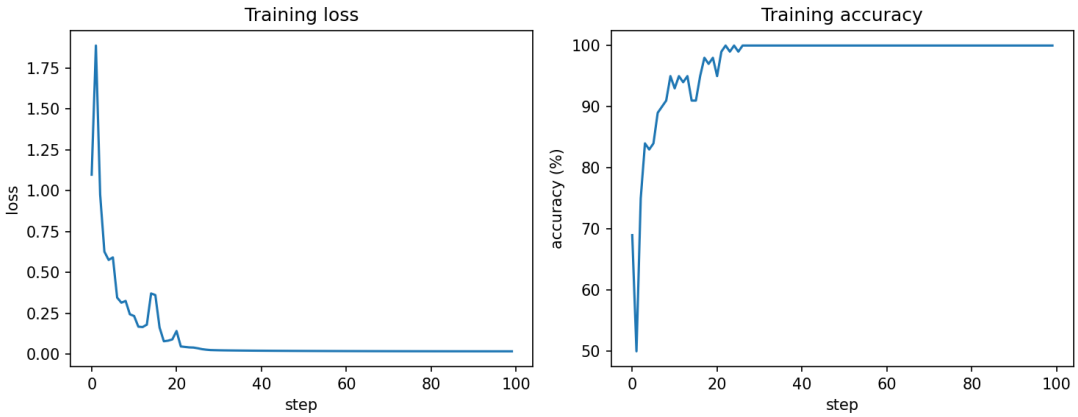


Figure 1: Training loss and accuracy.

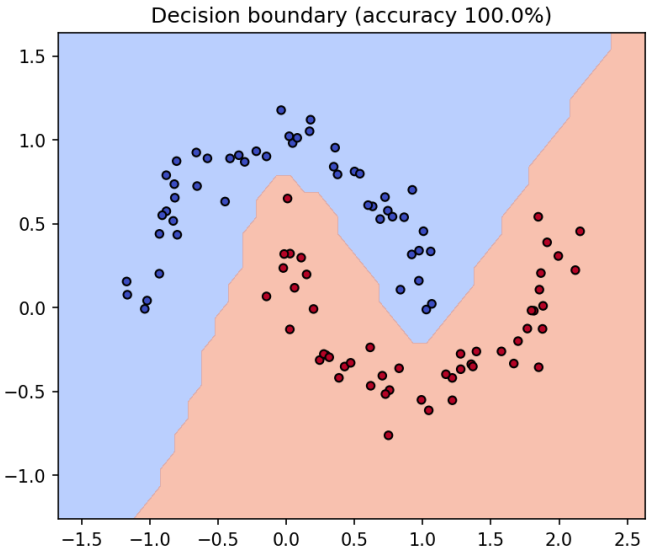


Figure 2: Learned decision boundary on the two moons data.

6 Design Justifications

- **Closures for `_backward`.** Each closure captures the exact operands and output of its own operation, so the backward pass needs no lookup table or type switch.

- **Composite operators.** Subtraction, negation and division reuse addition, multiplication and power. There is less code to get wrong, and their gradients are correct as soon as the three base rules are.
- **Strict operand types.** Rejecting non-numeric operands early turns silent errors into immediate `TypeError`s.
- **tanh derivative from the output.** Using $1 - z^2$ reuses the forward result and avoids recomputing exponentials.
- **Recursive DFS.** It matches the specification directly. Its limit is Python's recursion depth (about 1000 by default), which is far above the depth of the graphs used here.

7 Conceptual Questions

Q1. Why is backpropagation done in reverse topological order? A node can only pass on its gradient once that gradient is complete. Reverse topological order processes every consumer of a node before the node itself, so each `_backward` sees a final `out.grad`. Every node is visited once, so the backward pass costs about as much as the forward pass.

Q2. Why must gradients be accumulated with `+=`? By the multivariable chain rule, if a variable reaches the output through several paths, its derivative is the *sum* of the contributions of all paths. Plain assignment keeps only the last path. The diamond graph $d = (a + a) \cdot a$ gives 3 instead of 12 with `=`.

Q3. Why do we need the `visited` set? In a graph where a node is shared, depth-first search reaches the same node from several parents. Without the set the node is appended once per path. The work then grows exponentially with depth, and the same `_backward` is applied repeatedly, which counts contributions more than once.

Q4. Why must `zero_grad()` be called before each backward pass? Because gradients are accumulated, the `grad` fields still hold the previous step's values. Without zeroing, every update would use the sum of all past gradients instead of the current one.

Q5. Why are weights initialised randomly instead of to zero? If all weights in a layer start equal, all neurons compute the same output and receive the same gradient, so they stay identical forever. Random values break this symmetry. The range $[-1, 1]$ also keeps the pre-activations of a small network inside the region where tanh is not saturated, so the gradients do not vanish.

Q6. How does this engine differ from PyTorch? The algorithm is the same: reverse mode on a dynamically built graph. The difference is granularity. PyTorch nodes operate on whole tensors with vectorised kernels, while here every scalar multiply is a separate Python object. A layer with $n_{\text{in}} \times n_{\text{out}}$ weights creates that many nodes for each sample, which is why the engine is fine for teaching but far too slow for real models.

Q7. Why is reverse mode used rather than forward mode? A neural network maps many parameters to a single scalar loss. Reverse mode gives the derivatives with respect to *all* inputs in one backward pass. Forward mode would need one pass per parameter.